Cisco Secure Client Per-App VPN for Managed Mobile Devices (iOS/Android) via Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

How it's delivered . . . 4

What's included . . . 4

Scope boundary . . . 4

The bottom line . . . 4

Solution Overview 5

What this solution does . . . 5

The two moving parts (and the boundary between them) . . . 5

How per-app VPN works on iOS . . . 5

How per-app VPN works on Android . . . 6

On-demand behavior . . . 6

The managed-app-to-VPN association (the heart of it) . . . 6

Named objects this solution creates (reused verbatim) . . . 6

Where Conditional Access fits . . . 7

Outcome . . . 7

Requirements 7

R1. Intune enrollment for the target mobile devices . . . 7

R2. Managed apps (the apps that will tunnel) deployed by Intune . . . 8

R3. Cisco Secure Client app delivered as a managed app . . . 8

R4. A reachable Cisco VPN head-end with a supported auth method (Cisco side — deferred) . . . 8

R5. Authentication material — certificate profile or the chosen auth method . . . 8

R6. iOS: supervised / ADE enrollment + working APNs . . . 9

R7. Android: Android Enterprise enrollment + Managed Google Play . . . 9

R8. Entra security groups for targeting . . . 9

R9. A pilot group of real test users/devices . . . 9

R10. Change control and rollback (from global requirements) . . . 10

Implementation Guide 10

Phase 0 — Assess (read-only) . . . 10

Phase 1 — Configure the pilot targeting group . . . 11

Phase 2 — Deploy the client and managed apps to the pilot . . . 11

Phase 3 — Deploy authentication material (if certificate auth) . . . 11

Phase 4 — Build the per-app VPN profile for **iOS** . . . 11

Phase 5 — Build the per-app VPN profile for **Android** . . . 12

Phase 6 — Pilot . . . 13
Phase 7 — Validate (the pass/fail heart) . . . 13
Phase 8 — Expand . . . 13
Rollback . . . 14
Testing — Pass/Fail Gates 14
Gate 1 — Client and profile delivered . . . 14
Gate 2 — Authentication succeeds . . . 14
Gate 3 — Associated app tunnels (the primary gate) . . . 14
Gate 4 — Other traffic stays direct (the isolation gate) . . . 14
Gate 5 — On-demand behavior . . . 15
Gate 6 — Scope containment . . . 15
Gate 7 — Coverage report is sane . . . 15
Gate 8 — Rollback works . . . 15
Troubleshooting 15
Symptom 1 — VPN not triggering for the app . . . 16
Symptom 2 — All traffic tunneling (device-wide instead of per-app) . . . 16
Symptom 3 — App not associated . . . 17
Symptom 4 — Tunnel starts then fails to authenticate . . . 17
Symptom 5 — Profile not reaching the device at all . . . 18
Best Practices — The Managed-App Model 18
Keep the tunnel truly per-app . . . 18
Managed = tunneled: enforce the "managed" half . . . 18
Name everything, reuse verbatim . . . 18
Pilot first, expand in waves . . . 19
Authentication hygiene . . . 19
Respect the vendor boundary . . . 19
Align with Conditional Access . . . 19
Platform-specific notes . . . 19
Operate with the scripts . . . 19
Change control and rollback . . . 20
Deliverables Summary 20
Documentation . . . 20
PowerShell scripts . . . 20
Named objects produced . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

The problem

A device-wide mobile VPN backhauls **everything** a phone does — personal browsing, streaming, other apps — through the corporate network. That drains battery, adds latency, worries users about privacy, and expands what the corporate tunnel is responsible for. It is all-or-nothing when the business only needs *some* apps to reach internal resources.

The solution

Build **Cisco Secure Client per-app VPN** for **iOS** and **Android** through **Microsoft Intune** so that **only the named managed business apps** open the tunnel — on demand — and everything else stays direct. Intune delivers the client, the VPN profile, the authentication material, and the **app-to-VPN association**; the Cisco head-end (configured per Cisco's own documentation) terminates the tunnel.

How it's delivered

The repeatable model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and named, reused objects (`Cisco Per-App VPN – iOS`, `Cisco Per-App VPN – Android`, `PILOT – Per-App VPN Users`). It ships pilot-first and rollback-ready: prove that the right apps tunnel and everything else stays direct on a small pilot before touching the fleet.

What's included

- **Step-by-step guides** to build the per-app VPN profile for iOS and Android, associate the named managed apps, assign to a pilot, validate, and expand.
- **Five PowerShell scripts** (Microsoft Graph): readiness discovery, per-device validation, a coverage report, safe remediation of mis-scoped apps (preview-by-default), and a targeting-group builder.
- **Testing and troubleshooting** with clear gates — *managed apps tunnel; other traffic direct* — and best practices for the managed-app model.

Scope boundary

The Intune, Entra, and certificate side is documented at exact click-by-click detail. The **Cisco Secure Client mobile app and the Cisco VPN head-end** (gateway/firewall remote-access role, typically with Cisco ISE) are **deferred to Cisco's official documentation** — this solution consumes the head-end's server address and auth method rather than fabricating Cisco menu paths.

The bottom line

Only the business apps that need it tunnel to internal resources on managed mobiles — scoped by Intune, powered by Cisco Secure Client, and proven on a pilot before you scale. Self-guided, tested, and a fraction of the cost of a consulting engagement.

Solution Overview

What this solution does

This solution builds **Cisco Secure Client per-app VPN** for **iOS** and **Android** managed mobile devices using **Microsoft Intune**. Instead of a device-wide tunnel that backhauls every packet the phone sends, per-app VPN opens the Cisco Secure Client tunnel **only for the specific managed apps you associate with it**. Everything else — personal browsing, streaming, other apps — goes direct to the internet.

The result: your named business apps reach internal resources through the Cisco head-end, while the rest of the device stays off the corporate tunnel. Less backhaul, less battery drain, fewer privacy worries, and a smaller attack surface.

You deploy it the repeatable way — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first and rollback-ready, with named objects reused verbatim at every step.

The two moving parts (and the boundary between them)

Per-app VPN is a **partnership** between two control planes:

1. **Microsoft Intune (this solution's exact-detail scope).** Intune delivers the Cisco Secure

Client app, a **VPN profile** configured for per-app mode, the managed apps, and the **association** that binds each app to the VPN profile. Intune also delivers the authentication material (a certificate profile or the chosen auth method) and assigns everything to your Entra groups. This is where you click.

1. **The Cisco head-end (deferred to Cisco docs).** The Cisco Secure Client mobile app connects to a

Cisco VPN head-end (for example, a Cisco firewall/gateway with the remote-access VPN role, with policy typically brokered through Cisco ISE). The head-end's connection profile, address pools, split-tunnel/split-DNS rules, and any posture policy are configured **in Cisco's consoles** and are **deferred to Cisco's official documentation**. This solution names *what* you need from the head-end (a reachable connection entry / server address and a supported auth method) but does not fabricate Cisco menu paths.

> ***Where they meet:*** *Intune's VPN profile carries the* ***connection name / server address*** *that points at the Cisco head-end and the* ***per-app rules****. The Cisco head-end must* ***accept*** *that connection and the chosen authentication. Get the head-end reachable and authenticating first (per Cisco docs); this solution wires the Intune side to it precisely.*

How per-app VPN works on iOS

On iOS (and iPadOS), per-app VPN is a native platform capability:

• Intune deploys a **VPN profile** of connection type **Cisco Secure Client** (formerly AnyConnect),

configured for **per-app VPN** with **on-demand** enabled.

• You mark the profile to be used by specific **managed apps**. On iOS, apps must be

Intune-managed (installed/managed by MDM) for the association to take effect — a personally installed copy of the same app will **not** use the tunnel.

• When the user launches an associated app, iOS **starts the Cisco Secure Client tunnel on demand**

for that app's traffic and tears it down when the traffic stops. Traffic from unassociated apps never enters the tunnel.

- **Safari domain association** is available where you want specific web domains to trigger the tunnel inside Safari; the primary model here is app-to-VPN association.

iOS requirement to know now: reliable per-app VPN association is expected on **supervised** devices (enrolled via **Automated Device Enrollment / ADE**, i.e. Apple Business Manager). Working **APNs** (Apple Push Notification service) is required for Intune to manage iOS devices at all.

How per-app VPN works on Android

On **Android Enterprise**, per-app VPN is expressed through Intune as an **app-based VPN**:

- Intune deploys a **VPN profile** of connection type **Cisco Secure Client** on the Android Enterprise management profile (typically **work profile** for BYOD, or **fully managed** for corporate-owned).
- The profile carries an **allow-list of package names** — the apps *permitted* to use the VPN — (or, alternatively, a block-list of apps to exclude). Only the allow-listed managed apps send traffic through the Cisco Secure Client tunnel.
- **Always-on / on-demand behavior** is set on the profile. Android's per-app VPN is enforced by the work-profile boundary plus the package allow-list, so apps outside the list route directly.

Android requirement to know now: the device must be enrolled in **Android Enterprise** (work profile or fully managed) with **Managed Google Play** connected to Intune, so the Cisco Secure Client app and your business apps are delivered as **managed** apps referenced by package name.

On-demand behavior

"On-demand" means the tunnel is **not** a permanent always-connected device VPN. It is triggered by the associated app:

- **iOS:** on-demand rules start the tunnel when an associated app needs the network and stop it when idle — the user does not manually connect.
- **Android:** the app-based VPN engages for allow-listed apps; you choose whether it behaves always-on (for those apps) or connects as needed.

Either way, the **user experience is invisible for the right apps and untouched for everything else**.

The managed-app-to-VPN association (the heart of it)

The single most important concept: **an app only tunnels if it is both (a) managed by Intune and (b) associated with the VPN profile.** Miss either half and you get one of the two classic failures — the app doesn't tunnel (not associated / not managed), or *everything* tunnels (profile built device-wide instead of per-app). This solution names the exact apps, associates them explicitly, and validates that **only** those apps tunnel before expanding.

Named objects this solution creates (reused verbatim)

Object	Name	Where
iOS per-app VPN profile	`Cisco Per-App VPN – iOS`	Intune device configuration
Android per-app VPN profile	`Cisco Per-App VPN – Android`	Intune device configuration
Pilot targeting group	`PILOT – Per-App VPN Users`	Entra ID security group
Broad targeting group	`BROAD – Per-App VPN Users`	Entra ID security group
Certificate profile (if cert auth)	`Cisco Per-App VPN – User Cert`	Intune device configuration

Managed apps are referenced by their real store/package identity and named consistently as **"the associated managed apps"** throughout (you list yours in the implementation guide — e.g. an internal line-of-business app, a managed browser, a mail/intranet client).

Where Conditional Access fits

Per-app VPN scopes *network access* to specific apps; **Conditional Access (CA)** in Entra scopes *identity access* to specific apps and enforces device compliance. They align: put the same audience in `PILOT – Per-App VPN Users`, require **compliant, managed** devices in CA for the same business apps, and require the Intune device-compliance signal — so the app that tunnels is also the app that must come from a healthy, enrolled device. CA design detail is covered where it intersects the pilot; the network scoping is what this solution builds.

Outcome

At the end: named managed apps on iOS and Android open the Cisco Secure Client tunnel on demand to reach internal resources, all other traffic stays direct, and you have discovery/validation/reporting scripts plus a rollback to prove and maintain it — proven on `PILOT – Per-App VPN Users` before you expand to `BROAD – Per-App VPN Users`.

Requirements

Confirm every prerequisite below **before** you configure anything. Each entry follows the same shape — **Description / Why / Verify / Implement if missing** — so you can walk the list top to bottom and close gaps as you go. Category-wide items (Entra tenant, Intune, admin roles, change control, rollback) are in `global-requirements-network.md`; the items here are specific to Cisco per-app VPN on managed mobile devices.

> ***Vendor boundary:*** *the Cisco Secure Client mobile app and the Cisco VPN* ***head-end*** *(gateway/ firewall remote-access role, typically with Cisco ISE) are configured in* ***Cisco's consoles*** *and are* ***deferred to Cisco's official documentation****. Below you confirm *that* the head-end exists, is reachable, and authenticates your chosen method — not *how* to build it in Cisco.*

R1. Intune enrollment for the target mobile devices

Description: The iOS and Android devices that will run per-app VPN are enrolled in Microsoft Intune and compliant. **Why:** Per-app VPN profiles, the Cisco Secure Client app, and the app-to-VPN association are all delivered by Intune MDM. An unenrolled device cannot receive them. **Verify:** `https://intune.microsoft.com` → **Devices** → **All devices**; confirm the target iPhones/ iPads and Android devices appear as **enrolled** and **compliant**. **Implement if missing:** Assign an Intune-bearing license, then enroll — iOS via Apple enrollment (see R6), Android via Android Enterprise enrollment (see R7).

R2. Managed apps (the apps that will tunnel) deployed by Intune

Description: The specific business apps you want to tunnel are deployed and **managed** by Intune (not sideloaded / personally installed copies). **Why:** The association only engages for **Intune-managed** app instances. On iOS an unmanaged copy of the same app will bypass the tunnel; on Android the VPN allow-list references the **managed** package. **Verify:** Intune → **Apps** → **All apps**; confirm each target app exists as an assigned app (iOS store/VPP or LOB app; Android **Managed Google Play** app) and shows install/assignment to your users. **Implement if missing:** Intune → **Apps** → **All apps** → **Add** → choose the platform app type (iOS store app / iOS VPP / iOS LOB, or Android **Managed Google Play**), select the app, and assign it to `PILOT – Per-App VPN Users`. Record each app's **bundle ID (iOS)** / **package name (Android)** — you will need them for the association.

R3. Cisco Secure Client app delivered as a managed app

Description: The **Cisco Secure Client** app (iOS and Android) is deployed through Intune as a managed app to the target devices. **Why:** The VPN profile is configured for connection type Cisco Secure Client; the client app must be present and managed for the profile to bind and connect. **Verify:** Intune → **Apps** → **All apps**; confirm **Cisco Secure Client** exists for iOS (App Store / VPP) and for Android (**Managed Google Play**) and is assigned to your pilot group. **Implement if missing:** Add **Cisco Secure Client** from the App Store (iOS) and from Managed Google Play (Android) as managed apps, assign to `PILOT – Per-App VPN Users`, and set it **required** so it installs ahead of the VPN profile.

R4. A reachable Cisco VPN head-end with a supported auth method (Cisco side — deferred)

Description: A Cisco VPN **head-end** (gateway/firewall remote-access VPN role, policy typically via Cisco ISE) that the Cisco Secure Client can reach, with a **connection name / server address** and a **supported authentication method** (client certificate, or the method your head-end enforces). **Why:** Intune's VPN profile only points the client at the head-end and carries per-app rules; the head-end must accept the connection and authenticate it. Without a working head-end there is nothing to tunnel to. **Verify:** With Cisco Secure Client on a test device, confirm you can reach and authenticate to the head-end connection entry (following Cisco's docs). Record the exact **server address / connection name** and the **auth method** the head-end expects. **Implement if missing:** Stand up / configure the head-end connection profile, address pool, and split-tunnel/split-DNS **per Cisco's official documentation** (deferred here). This solution consumes its server address and auth method; it does not build the Cisco side.

R5. Authentication material — certificate profile or the chosen auth method

Description: The credential the client presents to the head-end. For certificate auth: a **trusted root certificate** profile and a **user/device SCEP or PKCS certificate** profile in Intune named `Cisco Per-App VPN – User Cert`. For other methods (e.g. federated / SAML at the head-end), the corresponding identity path. **Why:** The tunnel will not establish if the client cannot authenticate. Certificate auth is the common, silent, per-app-friendly choice and is delivered by Intune to the managed device. **Verify:** Intune → **Devices** → **Configuration** shows a **Trusted certificate** profile and a **SCEP/PKCS certificate** profile assigned to the same pilot group; on a test device the cert is present in the keychain/credential store. **Implement if missing:** Intune → **Devices** → **Configuration** → **Create** → **New policy** → platform → profile type **Trusted certificate** (deploy your issuing CA root), then a second policy of type **SCEP certificate** (or **PKCS certificate**) named `Cisco Per-App VPN – User Cert` pointing at your NDES/PKCS connector, and assign both to `PILOT – Per-App VPN Users`. Match the certificate's subject/ SAN and EKU to what the head-end expects (per Cisco docs).

R6. iOS: supervised / ADE enrollment + working APNs

Description: Target iOS/iPadOS devices are **supervised** (enrolled through **Automated Device Enrollment / ADE** via Apple Business Manager) and Intune has a valid **APNs** certificate. **Why:** Reliable per-app VPN association is expected on supervised devices; APNs is required for Intune to manage iOS at all. Without APNs, no iOS policy (including the VPN profile) is delivered. **Verify:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate** shows a valid, non-expired APNs certificate; target devices show **Supervised = Yes** under device details. **Implement if missing:** Upload/renew the **Apple MDM Push (APNs)** certificate; connect **Apple Business Manager** to Intune and (re)enroll target devices through **ADE** so they are supervised.

R7. Android: Android Enterprise enrollment + Managed Google Play

Description: Target Android devices are enrolled in **Android Enterprise** (work profile for BYOD or fully managed for corporate-owned) with **Managed Google Play** connected to Intune. **Why:** Android per-app (app-based) VPN and managed apps require the Android Enterprise management profile and package-name references from Managed Google Play. **Verify:** Intune → **Devices** → **Enrollment** → **Android** → **Managed Google Play** shows **connected**; target devices show an Android Enterprise ownership type (work profile / fully managed). **Implement if missing:** Intune → **Devices** → **Enrollment** → **Android** → **Managed Google Play** → **Connect** (bind your Managed Google Play account), then enroll devices into the appropriate Android Enterprise mode.

R8. Entra security groups for targeting

Description: Entra ID security groups used to scope assignment: a pilot group `PILOT – Per-App VPN Users` and a broad group `BROAD – Per-App VPN Users`. **Why:** Every profile, app, and certificate is assigned to a group. Piloting to a small group first is how you validate "only the right apps tunnel" before fleet rollout. **Verify:** Entra → **Groups**; both `PILOT – Per-App VPN Users` and `BROAD – Per-App VPN Users` exist. **Implement if missing:** Create them in Entra (**Groups** → **New group** → **Security**), or run `CPAVPN-Set-ScopeTargeting.ps1` (in this solution) to create/populate the pilot group. Start with a handful of known test users/devices in the pilot group.

R9. A pilot group of real test users/devices

Description: A small set of **real** pilot users on **real** enrolled iOS and Android devices, placed in `PILOT – Per-App VPN Users`. **Why:** Per-app VPN behavior (does the app tunnel? does other traffic stay direct?) must be observed on actual devices before expanding; simulators do not exercise the association reliably. **Verify:** `PILOT – Per-App VPN Users` contains at least one enrolled iOS device and one enrolled Android device belonging to cooperating testers. **Implement if missing:** Add cooperating testers (and/or their devices) to `PILOT – Per-App VPN Users`; confirm their devices are enrolled and compliant per R1.

R10. Change control and rollback (from global requirements)

Description: A change window and a written rollback: **un-assign** the per-app VPN profiles from the group (or remove the association) to revert to no-tunnel behavior; optionally remove the Cisco Secure Client app. **Why:** VPN/network changes affect how business apps reach internal resources and must be reversible. **Verify:** The rollback steps are written and approved before the pilot goes live (see G7 in `global-requirements-network.md`). **Implement if missing:** Document the rollback (remove profile assignment from `PILOT – Per-App VPN Users`) and a comms plan for testers.

Implementation Guide

Follow these steps in order. They implement the standard model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — for Cisco Secure Client per-app VPN on iOS and Android via Intune. Object names are fixed; reuse them **verbatim**.

> ***Vendor boundary:*** *the Cisco head-end (server address, connection profile, address pool, split-tunnel/split-DNS, posture) is configured in Cisco's consoles per* ***Cisco's official documentation*** *and is a prerequisite (see requirement R4). Below, you consume its* ***server address / connection name*** *and* ***auth method****; you do not build the Cisco side here.*

Throughout, "the associated managed apps" means the specific business apps you decided to tunnel. Fill in your real list before you start, keeping their identifiers handy:

Your app	iOS bundle ID	Android package name
(e.g. internal LOB app)	`com.yourorg.lob`	`com.yourorg.lob`
(e.g. managed browser)	`com.yourorg.browser`	`com.yourorg.browser`
(add rows as needed)	…	…

Phase 0 — Assess (read-only)

1. Run `CPAVPN-Discovery.ps1` (this solution) to inventory existing iOS/Android VPN profiles,

managed apps, and assignments, and to confirm your Entra groups exist. Read-only; makes no changes.

1. Confirm every requirement in `02-requirements.md` is green — especially **R4** (head-end reachable

and authenticating a test client) and **R5** (auth material / `Cisco Per-App VPN – User Cert`).

1. Confirm you have the head-end's exact **server address / connection name** and the **auth method**

it expects (from Cisco docs / your network team).

Gate: do not proceed until the Cisco Secure Client can reach and authenticate to the head-end on a single test device.

Phase 1 — Configure the pilot targeting group

1. Ensure `PILOT – Per-App VPN Users` exists and contains your real testers' enrolled devices. Create/

populate it in Entra (**Groups** → **New group** → **Security**) or run `CPAVPN-Set-ScopeTargeting.ps1`. Also create `BROAD – Per-App VPN Users` for later expansion.

Phase 2 — Deploy the client and managed apps to the pilot

1. In Intune → **Apps** → **All apps**, confirm (or add) **Cisco Secure Client** for **iOS** (App Store /

VPP) and **Android** (**Managed Google Play**), assigned **Required** to `PILOT – Per-App VPN Users` (requirement R3).

1. Confirm (or add) **each of your associated managed apps** for iOS (store/VPP/LOB) and Android

(Managed Google Play), assigned to `PILOT – Per-App VPN Users` (requirement R2). Note each app's **bundle ID / package name**.

Phase 3 — Deploy authentication material (if certificate auth)

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**: deploy your issuing CA **Trusted

certificate** root, assigned to `PILOT – Per-App VPN Users`.

1. Create a second policy of type **SCEP certificate** (or **PKCS certificate**) named

`Cisco Per-App VPN – User Cert`, matching the subject/SAN and EKU the head-end expects, assigned to `PILOT – Per-App VPN Users`. Confirm the cert lands on a test device before building the VPN profile.

Phase 4 — Build the per-app VPN profile for iOS

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.

• **Platform:** iOS/iPadOS

• **Profile type:** Templates → **VPN**

1. **Name:** `Cisco Per-App VPN – iOS`.

2. **Connection type: Cisco Secure Client** (formerly Cisco AnyConnect).

3. **Base VPN:**

• **Connection name:** a label users see (e.g. `Corp Per-App VPN`).

• **Server:** the head-end **server address / connection name** from R4 (set it as default).

• **Authentication method: Certificates**, and select the client certificate profile `Cisco Per-App VPN – User Cert` (or your chosen method to match the head-end).

1. **Automatic VPN** → **Type of automatic VPN: Per-app VPN**. (This is the switch that makes the tunnel app-scoped rather than device-wide — do **not** choose On-demand-only/device-wide here.)

1. **On-demand rules:** add a rule so the VPN **connects on demand** when an associated app needs the network (e.g. action **Connect** for the associated app domains/traffic). Leave device-wide on-demand off.

1. **Safari domains (optional):** if a web app should trigger the tunnel in Safari, add its domain(s).

2. **Assignments:** assign to `PILOT – Per-App VPN Users`. **Create.**

Associate the managed apps to the iOS profile

1. Associate each iOS managed app with `Cisco Per-App VPN – iOS` so **only those apps** use it:

• **On the VPN profile:** some tenants expose a **per-app VPN** association list directly on the profile — add each app's **bundle ID** there; **or**

• **On the app assignment:** Intune → **Apps** → **All apps** → **(the app)** → **Properties/Assignments** → edit the assignment for `PILOT – Per-App VPN Users` and set **VPN** = `Cisco Per-App VPN – iOS`. Do this for **every** app in your associated-apps table above, and **no others**.

Gate (iOS): each associated app now references `Cisco Per-App VPN – iOS`; unassociated apps reference **none**.

Phase 5 — Build the per-app VPN profile for Android

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.

• **Platform:** Android Enterprise

• **Profile type:** under the correct management mode (**Work profile** for BYOD, or **Fully managed / corporate-owned **for COBO/COPE)** → VPN**

1. **Name:** `Cisco Per-App VPN – Android`.

2. **Connection type: Cisco Secure Client** (Cisco AnyConnect).

3. **Base VPN:**

• **Connection name:** a label (e.g. `Corp Per-App VPN`).

• **Server:** the head-end **server address / connection name** from R4.

• **Authentication method: Certificates**, selecting `Cisco Per-App VPN – User Cert` (or your head-end's method).

1. **Per-app VPN (the association):** set the profile to apply to specific apps — add each associated app's **package name** to the **allowed apps** list (allow-list model), so only those managed apps route through the tunnel. (If your scenario needs the inverse, use the **excluded apps** list instead — but the allow-list is the per-app default here.)

1. **Always-on / on-demand:** configure the desired behavior for the allow-listed apps (on-demand / always-on for those apps only — not a device-wide always-on VPN).
1. **Assignments:** assign to `PILOT – Per-App VPN Users`. **Create.**

Gate (Android): the allow-list on `Cisco Per-App VPN – Android` contains **exactly** your associated package names — no more, no less.

Phase 6 — Pilot

1. Let the pilot devices sync (or trigger **Sync** from Intune / Company Portal). Confirm on each pilot device:

- **Cisco Secure Client** installed and managed,
- the certificate present (if cert auth),
- `Cisco Per-App VPN – iOS` / `Cisco Per-App VPN – Android` profile installed.

1. Run `CPAVPN-Validation.ps1 -DeviceName your-pilot-device` per pilot device to confirm the per-app VPN profile is present and the app association state is correct (Graph, read-only).

Phase 7 — Validate (the pass/fail heart)

1. On a pilot device, **launch an associated app** and access an internal resource. It should connect **through the Cisco Secure Client tunnel** (on-demand). **PASS** = associated app reaches internal resource via the tunnel.
1. On the same device, use an **unassociated app / general browsing** to a public site. It must go **direct** (no tunnel). Confirm via the head-end's connected-client / session view or by checking the app's egress. **PASS** = other traffic is direct, not tunneled.
1. Work the full pass/fail matrix in `05-testing.md`. If **all** traffic tunnels, or the app does **not** tunnel, stop and use `06-troubleshooting.md` before expanding.

Phase 8 — Expand

1. Populate `BROAD – Per-App VPN Users` (via `CPAVPN-Set-ScopeTargeting.ps1` or Entra) in waves.
2. Add `BROAD – Per-App VPN Users` to the **Assignments** of `Cisco Per-App VPN – iOS`, `Cisco Per-App VPN – Android`, the certificate profile, the Cisco Secure Client app, and each associated managed app.
1. Run `CPAVPN-Report.ps1` to measure per-app VPN coverage across managed mobile devices; expand wave by wave until the fleet is covered.

Rollback

To revert: remove `PILOT – Per-App VPN Users` (and `BROAD – Per-App VPN Users`) from the **Assignments** of `Cisco Per-App VPN – iOS` and `Cisco Per-App VPN – Android` (or remove the app↔VPN association). Devices drop the profile on next sync and return to no-tunnel behavior. Optionally remove the Cisco Secure Client app. Use `CPAVPN-Remediation.ps1` (default `-WhatIf`) to preview any association fixes before applying.

Testing — Pass/Fail Gates

Test on **real** enrolled pilot devices in `PILOT – Per-App VPN Users` — at least one **iOS** and one **Android** device. Each gate has an explicit **PASS** and **FAIL**. Do not expand to `BROAD – Per-App VPN Users` until every gate passes on both platforms.

> *The core acceptance criterion for this whole solution: **the associated managed apps tunnel; everything else stays direct.***

Gate 1 — Client and profile delivered

Test: On a pilot device, confirm **Cisco Secure Client** is installed/managed and the profile `Cisco Per-App VPN – iOS` (or `Cisco Per-App VPN – Android`) is installed. Optionally run `CPAVPN-Validation.ps1 -DeviceName your-pilot-device`. **PASS:** Client present and managed; the named per-app VPN profile is present on the device. **FAIL:** Client missing/unmanaged, or the profile absent → check app assignment (R3), enrollment (R1), and iOS APNs / Android Managed Google Play (R6/R7). Return to `03-implementation.md` Phase 2.

Gate 2 — Authentication succeeds

Test: Trigger the tunnel (launch an associated app) and confirm it authenticates to the head-end. **PASS:** The tunnel establishes; the head-end shows a connected session for the test user/device. **FAIL:** Auth error / no session → verify the certificate profile `Cisco Per-App VPN – User Cert` landed and its subject/SAN/EKU match what the head-end expects; confirm server address (R4/R5). Cisco head-end auth policy is deferred to Cisco docs.

Gate 3 — Associated app tunnels (the primary gate)

Test: Launch an **associated managed app** and reach an **internal resource** (intranet host, internal API, file share URL) that is only reachable via the corporate network. **PASS:** The internal resource loads **through the Cisco Secure Client tunnel** (on-demand), with a corresponding session on the head-end. **FAIL:** Internal resource unreachable / app not tunneling → the app is likely **not associated** or **not managed**. Confirm the app is Intune-managed and appears in the association (iOS) / allow-list (Android). See `06-troubleshooting.md` → "VPN not triggering for the app."

Gate 4 — Other traffic stays direct (the isolation gate)

Test: On the **same** device, browse a public site or use an **unassociated** app. **PASS:** The traffic goes **direct** — **no** tunnel session appears on the head-end for that traffic, and the device's public egress is its

normal internet path, not the corporate egress. **FAIL:** Public/other traffic is tunneling → the profile is behaving **device-wide**, not per-app. Confirm iOS **Automatic VPN = Per-app VPN** (not device-wide on-demand) and the Android **allow-list** contains only your apps. See `06-troubleshooting.md` → "All traffic tunneling."

Gate 5 — On-demand behavior

Test: Close the associated app / let it idle; then reopen it. **PASS:** The tunnel starts on demand when the associated app needs the network and does not persist as an always-connected device VPN for unrelated apps. **FAIL:** Tunnel never starts (on-demand rule missing) or is always-on device-wide (wrong mode) → revisit on-demand rules (iOS) / always-on-per-app settings (Android) in Phase 4/5.

Gate 6 — Scope containment

Test: Confirm a device/user **outside** `PILOT – Per-App VPN Users` receives **no** per-app VPN profile. **PASS:** Only pilot members have the profile; non-members are untouched. **FAIL:** Non-pilot devices got the profile → check assignment targeting; ensure only `PILOT – Per-App VPN Users` is assigned during pilot.

Gate 7 — Coverage report is sane

Test: Run `CPAVPN-Report.ps1`. **PASS:** Reported per-app VPN coverage matches the pilot membership (right devices covered, right apps associated); no unexpected devices. **FAIL:** Coverage over/under-reaches expectation → reconcile assignments and associations, re-run.

Gate 8 — Rollback works

Test: In a controlled window, remove `PILOT – Per-App VPN Users` from the profile assignments; sync a pilot device. **PASS:** The profile is removed on next sync; associated apps stop tunneling and behave as normal (direct) apps. **FAIL:** Profile lingers → force sync / retire-and-re-check; confirm assignment removal saved.

Exit criteria (all must be true, on both iOS and Android)

- Associated managed apps reach internal resources through the tunnel (Gate 3).
- All other traffic is direct (Gate 4).
- On-demand behavior is correct (Gate 5).
- Scope is contained to the pilot group (Gate 6) and the coverage report is accurate (Gate 7).
- Rollback is proven (Gate 8).

Only then assign `BROAD – Per-App VPN Users` and expand in waves.

Troubleshooting

Symptom-first fixes for Cisco Secure Client per-app VPN on managed iOS/Android via Intune. Work the Intune side precisely; where a cause is clearly **Cisco head-end** (routing, address pool, posture, ISE

policy), that is **deferred to Cisco's official documentation** — this guide points you there but does not fabricate Cisco menu paths.

Run `CPAVPN-Validation.ps1 -DeviceName your-device` and `CPAVPN-Discovery.ps1` first — they surface most of these causes read-only.

Symptom 1 — VPN not triggering for the app

The associated app opens but does **not** reach the internal resource / no tunnel starts.

Check, in order:

1. **Is the app Intune-managed?** The association only works for the **managed** instance. On iOS a

personally installed copy will not tunnel. Confirm the app appears under Intune → **Apps** → **All apps** assigned to `PILOT – Per-App VPN Users`, and that the copy on the device is the managed one (requirement R2).

1. **Is the app actually associated?** Confirm the app references `Cisco Per-App VPN – iOS`

(iOS: on the app assignment's **VPN** field or the profile's per-app list) or is in the `Cisco Per-App VPN – Android` **allow-list** by exact **package name** (Android). A typo'd bundle ID / package name silently drops the association. `CPAVPN-Validation.ps1` lists the associated apps.

1. **Is the profile applied to the device?** `CPAVPN-Validation.ps1` should show the profile present.

If not, check assignment to `PILOT – Per-App VPN Users`, enrollment (R1), and iOS APNs / Android Managed Google Play (R6/R7). Force a **Sync**.

1. **On-demand rule missing (iOS).** If the profile is present but nothing connects, confirm the

On-demand rule exists so the tunnel connects when the associated app needs the network (Implementation Phase 4, step 14).

1. **Client not installed.** Confirm **Cisco Secure Client** is installed and managed (R3) — the

profile can't bind without it.

1. **Auth failing at the head-end.** If the tunnel starts then drops, this is usually authentication —

see Symptom 4.

Symptom 2 — All traffic tunneling (device-wide instead of per-app)

Public browsing and unassociated apps are going through the tunnel — the isolation gate (Testing Gate 4) fails.

Check, in order:

1. **iOS profile is device-wide, not per-app.** In `Cisco Per-App VPN – iOS`, confirm **Automatic

VPN → Type of automatic VPN = Per-app VPN **—** not** a device-wide On-demand VPN. A device-wide on-demand rule with a broad match will tunnel everything. Rebuild the automatic-VPN section per Implementation Phase 4, step 13.

1. **Android allow-list too broad / inverted.** In `Cisco Per-App VPN – Android`, confirm you used the

allowed apps (allow-list) containing only your package names — not an empty allow-list (which can behave as "all"), and not an **excluded-apps** model that leaves everything else tunneling. Fix the list (Implementation Phase 5, step 22).

1. **Always-on device VPN set.** Confirm you did not enable a device-wide **always-on** VPN. Per-app

on-demand should engage per app, not for the whole device.

1. **Split-tunnel at the head-end (Cisco side — deferred).** If the profile is correctly per-app but

the *associated* app's traffic is pulling in more than expected, the head-end's split-tunnel / split-DNS policy may be sending broad routes. Review the head-end's tunnel policy **per Cisco docs**.

Symptom 3 — App not associated

The app tunnels for no one, or the association simply won't stick / doesn't appear.

Check, in order:

1. **Exact identifier mismatch.** The association keys on the **bundle ID (iOS)** / **package name

(Android)**. Confirm the identifier on the association exactly matches the app's real identifier (from `CPAVPN-Discovery.ps1`'s managed-apps list). One character off = no association.

1. **Association set in the wrong place (iOS).** iOS association can live on the **VPN profile's**

per-app list **or** on the **app assignment's VPN field** — set it in the place your tenant exposes, and don't split it across both inconsistently (Implementation Phase 4, step 17).

1. **App not assigned to the same group.** The app and the VPN profile must both target

`PILOT – Per-App VPN Users`. If the app is assigned to a different group than the profile, the device may have one without the other.

1. **Use remediation to reconcile.** Run

`CPAVPN-Remediation.ps1 -IntendedApps your-bundle-id,your-package-name` (default `-WhatIf`) to see the ADD/REMOVE diff between the intended app set and what's currently associated, then re-run with `-Apply`.

Symptom 4 — Tunnel starts then fails to authenticate

Check, in order:

1. **Certificate present?** Confirm `Cisco Per-App VPN – User Cert` (and the trusted root) deployed to

the device (R5). An expired or missing client cert breaks every associated app at once.

1. **Cert subject/SAN/EKU mismatch.** The head-end expects a specific certificate shape. Match the

Intune SCEP/PKCS profile's subject/SAN and EKU to the head-end's expectation (**per Cisco docs**).

1. **Wrong server / connection name.** Confirm the **Server** value in the VPN profile is the correct

head-end **server address / connection name** from R4.

1. **Head-end policy (Cisco side — deferred).** If Intune-side auth material is correct but the

head-end rejects the session, the connection profile / ISE authorization policy is the place to look — **deferred to Cisco documentation**.

Symptom 5 — Profile not reaching the device at all

1. **iOS:** check **APNs** certificate validity and that the device is **supervised (ADE)** (R6);

without APNs no iOS policy is delivered.

1. **Android:** check **Managed Google Play** is connected and the device is enrolled in Android

Enterprise (R7).

1. **Assignment / compliance:** confirm the device's user/device is in `PILOT – Per-App VPN Users` and

the device is compliant (R1). Force **Sync** from Company Portal / Intune.

When to escalate to the Cisco side

If the Intune profile is correct (per-app mode, right association, right server, cert present) but tunnels still misbehave — broad routing, posture blocks, address-pool exhaustion, or authorization rejects — the cause is on the **Cisco head-end / ISE**. Escalate there and follow **Cisco's official documentation**; keep the Intune profile's server address in sync with any head-end change.

Best Practices — The Managed-App Model

Guidance for running Cisco Secure Client per-app VPN on managed iOS/Android well, over time.

Keep the tunnel truly per-app

• **Never** promote the profile to device-wide to "make it work." If an app won't tunnel, fix the

association, not the scope. A device-wide VPN reintroduces the backhaul, battery, and privacy problems this solution exists to solve.

• On iOS, always keep **Automatic VPN = Per-app VPN** with **on-demand**; on Android keep the

allow-list tight to named packages. Review both whenever you touch the profile.

Managed = tunneled: enforce the "managed" half

• The association only works for **Intune-managed** app instances. Deploy every business app that

should tunnel **through Intune** and, where possible, block/prevent unmanaged copies (e.g. require managed-app install, use app protection policies) so users can't accidentally run an untunneled personal copy.

• Keep an authoritative list of the **associated apps** (bundle IDs / package names). Treat it as the

contract: an app is only in scope if it is on this list *and* managed.

Name everything, reuse verbatim

- Use `Cisco Per-App VPN – iOS`, `Cisco Per-App VPN – Android`, `Cisco Per-App VPN – User Cert`,

`PILOT – Per-App VPN Users`, `BROAD – Per-App VPN Users` exactly and consistently. Consistent names make the discovery, validation, report, and remediation scripts reliable and make audits trivial.

Pilot first, expand in waves

- `PILOT – Per-App VPN Users` → validate all gates in `05-testing.md` → only then

`BROAD – Per-App VPN Users`, in waves. Run `CPAVPN-Report.ps1` between waves to confirm coverage tracks expectation.

Authentication hygiene

- Prefer **certificate authentication** (silent, per-app-friendly) via `Cisco Per-App VPN – User

Cert`. Match subject/SAN and EKU to the head-end's expectation (per Cisco docs) and monitor **certificate expiry/renewal** — an expired cert silently breaks the tunnel for every associated app at once.

- Keep the **trusted root** deployed alongside the client cert so chain validation never fails.

Respect the vendor boundary

- The Cisco head-end (connection profile, address pool, split-tunnel/split-DNS, posture, ISE policy)

is **Cisco's domain** — change it in Cisco's consoles per **Cisco's official documentation**, and keep the Intune profile's **server address / connection name** in sync with it. When a tunnel issue is clearly head-end-side (routing, posture, address pool), escalate to the Cisco side rather than editing Intune blindly.

Align with Conditional Access

- Put the same audience in `PILOT – Per-App VPN Users` and in the **Conditional Access** policy that

requires **compliant, managed** devices for the business apps. Then the app that tunnels is also the app that must originate from a healthy, enrolled device — network scope and identity scope reinforce each other.

Platform-specific notes

- **iOS:** keep target devices **supervised (ADE)** and **APNs** healthy — per-app association is most

reliable there, and APNs is the delivery lifeline for all iOS policy.

- **Android:** keep **Managed Google Play** connected and reference apps by **package name**. Validate

the allow-list after any app repackaging or package-name change.

Operate with the scripts

- Run `CPAVPN-Discovery.ps1` before changes (read-only baseline), `CPAVPN-Validation.ps1` per device

when investigating, `CPAVPN-Report.ps1` on a schedule for coverage, and `CPAVPN-Remediation.ps1` (default `-WhatIf`) to preview any association fix before applying it. Always preview, then apply.

Change control and rollback

- Make VPN/association changes in a **change window** with the rollback ready (un-assign the profile

from the group). Communicate to affected users, since it changes how business apps reach internal resources.

Deliverables Summary

Everything included in this solution, and what each piece does.

Documentation

Document	Purpose
Solution Overview (`01-solution-overview.md`)	How per-app VPN works on iOS and Android, the managed-app-to-VPN association, on-demand behavior, and where Intune's per-app VPN config meets the Cisco head-end.
Requirements (`02-requirements.md`)	Every prerequisite with Description / Why / Verify / Implement-if-missing — Intune enrollment + managed apps, Cisco Secure Client app + VPN profile, certificate/auth method, Entra groups, pilot group, iOS supervised/ADE + APNs, Android Enterprise.
Implementation Guide (`03-implementation.md`)	Exact ordered steps: build the per-app VPN profile for iOS and Android, associate the named managed apps, assign to the pilot, validate only those apps tunnel, expand.
Testing (`05-testing.md`)	Pass/fail gates — associated apps tunnel, other traffic direct, on-demand, scope containment, rollback.
Troubleshooting (`06-troubleshooting.md`)	Fixes for VPN not triggering for the app, all traffic tunneling, and app not associated.
Best Practices (`07-best-practices.md`)	Running the managed-app model well over time.
Global Requirements — Network (`global-requirements-network.md`)	Category-wide prerequisites (Entra, Intune, admin roles, pilot group, change control/rollback).
Executive Summary (`executive-summary.md`)	One-page business framing for decision-makers.

PowerShell scripts

All scripts are **read-only** against Microsoft Graph except the two remediation/targeting tools, which default to `-WhatIf` (preview) and change nothing until you opt in. Cisco head-end internals are deferred to Cisco's documentation.

Script	What it does	Default safety
`CPAVPN-Discovery.ps1`	Per-app VPN readiness: enumerates iOS/Android VPN profiles, managed apps, and assignments; confirms the Entra scope groups exist.	Read-only
`CPAVPN-Validation.ps1`	For a named managed device, confirms the per-app VPN profile is present and reports the app↔VPN association state.	Read-only
`CPAVPN-Report.ps1`	Per-app VPN coverage across managed mobile devices — which devices carry the profile, which apps are associated.	Read-only
`CPAVPN-Remediation.ps1`	Safely previews/fixes mis-scoped app/VPN associations (e.g. an app that should/shouldn't be associated).	`-WhatIf` by default
`CPAVPN-Set-ScopeTargeting.ps1`	Creates/populates the pilot (and broad) Entra targeting group(s).	`-WhatIf` by default

Required Microsoft Graph scopes

- **Discovery / Validation / Report (read-only):** `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Group.Read.All`.
- **Remediation / Set-ScopeTargeting (write):** `DeviceManagementConfiguration.ReadWrite.All`, `DeviceManagementApps.ReadWrite.All`, `Group.ReadWrite.All`, `GroupMember.ReadWrite.All`.

Named objects produced

Object	Name
iOS per-app VPN profile	`Cisco Per-App VPN – iOS`
Android per-app VPN profile	`Cisco Per-App VPN – Android`
Client certificate profile (if cert auth)	`Cisco Per-App VPN – User Cert`
Pilot targeting group	`PILOT – Per-App VPN Users`

Object	Name
Broad targeting group	`BROAD – Per-App VPN Users`

Outcome

Named managed business apps on iOS and Android open the Cisco Secure Client tunnel on demand to reach internal resources; all other traffic stays direct — proven on the pilot, reported across the fleet, and rollback-ready.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CPAVPN-Discovery.ps1

Purpose: Inventories per-app VPN readiness in Microsoft Intune: enumerates iOS and Android VPN device-configuration profiles (flagging Cisco Secure Client / per-app ones), lists managed mobile apps and their bundle IDs / package names, shows profile and app assignments, and confirms the Entra targeting groups exist. Read-only baseline before any change. Cisco head-end configuration is deferred to Cisco documentation. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPAVPN-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Discovery.ps1

CPAVPN-Remediation.ps1

Purpose: Previews and (only when you opt in) fixes app-to-VPN association scope for the Cisco per-app VPN profiles: ensures the intended managed apps are assigned to use the per-app VPN and flags apps that carry the association but should not. Defaults to -WhatIf so nothing changes until you pass -Apply. Cisco head-end changes are out of scope and deferred to Cisco documentation. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPAVPN-Remediation.ps1 -IntendedApps com.yourorg.lob,com.yourorg.browser Apply the fix: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Remediation.ps1 -IntendedApps com.yourorg.lob -Apply

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPAVPN-Remediation.ps1 -IntendedApps com.yourorg.lob,com.yourorg.browser Apply the fix: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Remediation.ps1 -IntendedApps com.yourorg.lob -Apply

CPAVPN-Report.ps1

Purpose: Produces a coverage report for Cisco per-app VPN across managed iOS and Android devices: which devices carry the 'Cisco Per-App VPN - iOS' / 'Cisco Per-App VPN - Android' profile, the applied/not-applied state, and a rollup by platform. Optionally writes a CSV. Read-only. Cisco head-end coverage is confirmed on the Cisco side, not here. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPAVPN-Report.ps1 -CsvPath .\cpavpn-coverage.csv

How to run: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Report.ps1 -CsvPath .\cpavpn-coverage.csv

CPAVPN-Set-ScopeTargeting.ps1

Purpose: Creates the Entra ID security groups that scope Cisco per-app VPN assignment — 'PILOT - Per-App VPN Users' and (optionally) 'BROAD - Per-App VPN Users' — and populates the pilot group with the members you name. Defaults to -WhatIf so nothing changes until you pass -Apply. Assign the VPN profiles and apps to these groups per the Implementation Guide. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPAVPN-Set-ScopeTargeting.ps1 -PilotMembers user1@your-tenant.com,user2@your-tenant.com Apply: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Set-ScopeTargeting.ps1 -PilotMembers user1@your-tenant.com -Apply

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CPAVPN-Set-ScopeTargeting.ps1 -PilotMembers user1@your-tenant.com,user2@your-tenant.com Apply: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Set-ScopeTargeting.ps1 -PilotMembers user1@your-tenant.com -Apply

CPAVPN-Validation.ps1

Purpose: For one named managed mobile device, confirms the Cisco per-app VPN profile ('Cisco Per-App VPN - iOS' / 'Cisco Per-App VPN - Android') is assigned/applied and reports the app-to-VPN association state — which managed apps are in scope to tunnel. Read-only, per device. Cisco head-end session/posture state is confirmed on the Cisco side, not here. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPAVPN-Validation.ps1 -DeviceName your-pilot-device

How to run: powershell -ExecutionPolicy Bypass -File .\CPAVPN-Validation.ps1 -DeviceName your-pilot-device